

ENGINEERING BENCHMARK REPORT

Blacksmith vs. GitHub Actions

A CI Runner Benchmark on a Real-World Workload

Test subject: a fork of **immich-app/immich** — a production-scale TypeScript monorepo — running identical CI jobs on GitHub-hosted runners and on Blacksmith runners, side by side.

Benchmark runs executed July 8, 2026 · 28 recorded job-runs · 6 CI jobs · cold and warm cache states

1.4–5.3×

Blacksmith speedup range — faster on every job tested

5.3×

largest single-step gap: the ESLint step (390s → 73s)

14 / 14

job-runs per platform; zero flakiness observed on either

n = 1–2

runs per cell — a directional pilot, not a powered study

1 Executive Summary

Blacksmith was faster than GitHub-hosted runners on **every one of the six CI jobs tested**, with speedups ranging from roughly **1.4×** (the e2e suite) to **5.3×** (the ESLint step of the web-lint job). The pattern was consistent across cold and warm cache states — this is not a caching artifact.

The standout finding is **where** the gap is largest: ESLint. The web-lint job's "Run linter" step took 390s on GitHub Actions versus 73s on Blacksmith. ESLint's TypeScript-aware rules are largely single-threaded, CPU-bound work — exactly the profile where Blacksmith's bare-metal, single-core-performance pitch would predict its biggest gains, and that is precisely where they showed up.

The speed comes with a real tradeoff: **setup friction**. GitHub Actions is the zero-setup default for any repository. Blacksmith required creating a GitHub organization (signup does not work against a personal account), waiting overnight for human approval of its GitHub App, and editing every workflow to swap runner labels and cache/build actions. None of it is hard; none of it is free.

Reliability was a tie at this sample size: 14 job-runs attempted per platform, exactly one failure each — and it was the same failure, a bug in our own workflow (a missing git submodule), not a platform fault. No flakiness, cache corruption, or stuck runs were observed on either side. Debugging and observability were essentially identical, because Blacksmith is a runner substitution inside GitHub Actions, not a different CI product.

Read this with the right confidence level

This report is an expanded pilot: 1–2 runs per job/cache-state cell, executed in a single session on July 8, 2026. The findings are directional and internally consistent, but they are not p50/p95-grade statistics. The originally planned 20–30-runs-per-cell study would be needed to make them so (Section 10).

2 Methodology

The workload is a fork of **immich-app/immich**, the open-source photo-management platform — a large, real-world TypeScript monorepo with a substantial CI surface, hosted under a purpose-made GitHub organization (*Wanderment-Farms*). Two branches carry identical benchmark workflows that differ only in runner platform: **bench/gha** (GitHub-hosted runners) and **bench/blacksmith** (Blacksmith runners plus its equivalent cache/build actions). All timings are from real GitHub Actions runs; run IDs and URLs for every data point are preserved in the raw results file (*pilot-results.csv*).

Six benchmark jobs

- **server-unit-tests** — server lint, typecheck, and the full Vitest unit suite (2,237 tests)
- **server-medium-tests** — integration-grade server tests against real test assets
- **web-lint** — web formatting check, ESLint, and svelte/ts checks
- **web-unit-tests** — web Vitest unit suite

- **e2e-tests-server-cli** — end-to-end server + CLI tests
- **docker-build-server** — server Docker image build (build-only, no push)

Each job was run on both platforms in cold-cache and warm-cache states. A deliberate test failure was also pushed to both branches to compare the debugging experience (Section 7).

Scope disclosure

This study began as a time-boxed, under-one-hour pilot (one job, $n=1$ cold + warm per platform) and was expanded in the same session to the full six-job matrix once Blacksmith access came through. Sample sizes remain $n=1-2$ per cell — short of the 20–30 runs per cell the original design called for. One further caveat: the benchmark workflow was written from scratch as a minimal *bench.yml* (+70 lines, one new file), not a line-by-line migration of Immich's production *test.yml* — so the setup-effort numbers understate what a true migration of an existing workflow would involve.

3 Setup & Onboarding

Elapsed time from first workflow commit to a working benchmark: **about 40 minutes** (02:48–03:28 UTC), covering the workflow itself, two debugging detours, and an unplanned repository-ownership transfer. Four issues were hit along the way:

- **Immich's real CI is not portable to a fork.** It depends on an org-only GitHub App token and self-hosted runner labels (*mich*, *pokedex-large*). Known going in; the minimal bench workflow works around it.
- **A phantom toolchain dependency.** The `//:plugins` target unexpectedly pulled in the extism-js WASM toolchain via *plugin-core*, which the server does not actually depend on — dropped from the install/build scope.
- **GitHub dispatch rules.** *workflow_dispatch*-only workflows cannot be dispatched unless the file exists on the default branch. Fixed with a *push* trigger scoped to each bench branch — after a safety check correctly blocked an initial attempt to push workflow changes to *main*.
- **Blacksmith requires organization ownership.** Its signup will not attach to a personal account — discovered mid-signup, not from the docs. Both repos had to be transferred from a personal account to a new org mid-experiment, which auto-cancelled two already-queued Blacksmith runs (re-triggered afterward). The subsequent GitHub App install then sat in Blacksmith's human-approval queue overnight — the single largest source of dead time in the whole exercise.

One documentation note: Blacksmith's quickstart and migration guide were clear and accurate about the mechanical *runs-on* swap, but did not surface the org-ownership requirement, and its in-product “Migration Wizard” defaults to auto-generating a PR that rewrites *every* workflow in the repo — fine for a real one-time migration, actively wrong for a controlled side-by-side comparison. Caught before it caused damage, but only through close supervision.

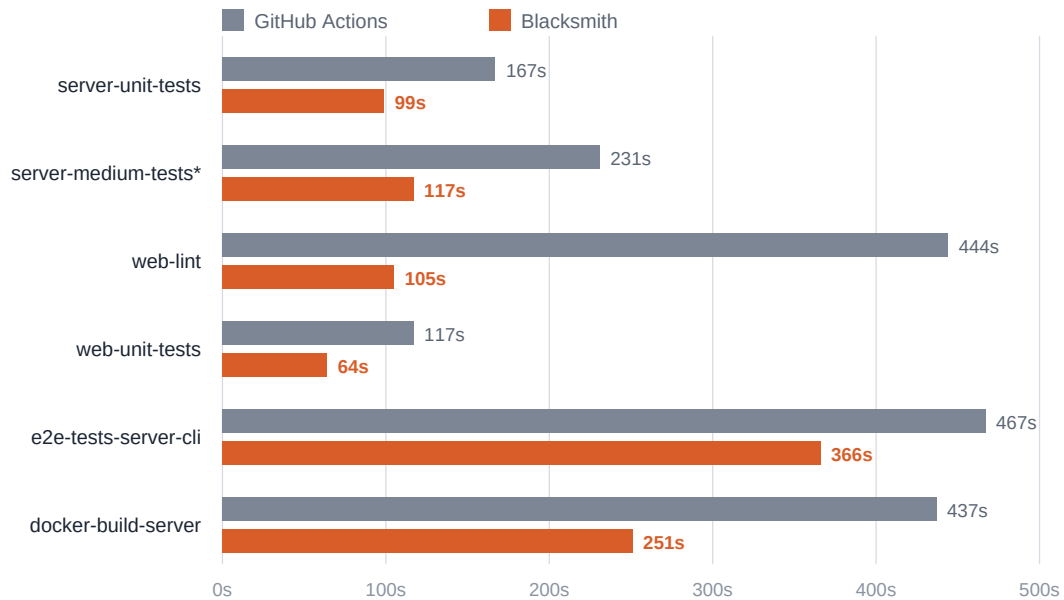
Verdict: GitHub Actions wins this dimension. Blacksmith's friction is real but mostly one-time; GitHub's zero-setup default asks for none of it.

4 Workflow Performance

Blacksmith was faster on every job, in every cache state. The table shows job duration in seconds (ranges where multiple runs exist; n=1–2 per cell).

Job	GHA cold (s)	GHA warm (s)	Blacksmith cold (s)	Blacksmith warm (s)	Speedup
server-unit-tests	167–208	200–205	88–99	83–101	~1.7–2.0×
server-medium-tests	209†	231	87†	117	~2.0×
web-lint	444	435	105	99	~4.2–4.4×
web-unit-tests	117	140	64	61	~1.8–2.3×
e2e-tests-server-cli	467	474	366	290	~1.3–1.6×
docker-build-server	437	18	251	17	1.7× cold / tie warm

† First server-medium-tests run failed identically on both platforms due to a bug in our own workflow (missing git submodule — see Section 5); durations shown alongside are the post-fix passing runs.



Cold-cache job duration, full six-job suite (one run per bar — directional). *server-medium-tests bars are the post-fix passing runs.

Standout finding — ESLint is where Blacksmith's edge is biggest

The web-lint job's "Run linter" step alone: **390s on GitHub Actions vs. 73s on Blacksmith — a 5.3× difference**, far larger than the ~2× pattern on every other job, and consistent across both the cold and warm run (444s/435s vs. 105s/99s total job time) — so it is not a cache artifact. ESLint's TypeScript-aware rules are largely single-threaded, CPU-bound work: exactly the profile where a bare-metal, high-single-core-performance runner should show its largest gains. This is the most platform-differentiated single step found in the entire benchmark.

Where the platforms do not differ: Docker layer caching. Cold-to-warm rebuild went 437s → 18s on GitHub Actions (*type=gha* cache) and 251s → 17s on Blacksmith (automatic NVMe cache) — both essentially instant on a no-source-change rebuild. Cache effectiveness is a wash; only the cold build differentiates.

Also worth noting: warm caches barely helped the compute-bound jobs on either platform (e.g. server-unit-tests, ~8s cold-to-warm delta on GHA). These jobs are dominated by lint/typecheck/test compute, not dependency installation — which is precisely why raw runner speed, not caching, is the differentiator in this suite.

5 Two Incidents That Stress-Tested the Benchmark

Two things went wrong mid-benchmark. Both were caught and fixed, and both are worth recording — they are evidence the investigation was adversarial about its own setup, not a rubber stamp.

Caught before it shipped: an unrequested public artifact

While drafting the Docker build job, one automation agent defaulted the build to *push: true* against *ghcr.io/wanderment-farms/immich-server* — a real public container image nobody asked for. A safety check blocked it on the Blacksmith branch before it landed; the GitHub Actions branch had already started pushing when caught, and that in-flight run was cancelled mid-push (its push step shows *cancelled*, not *success*). Both branches were fixed to build-only (*push: false*). Benchmark jobs should never have side effects; this one nearly did.

A real bug, found and fixed on both platforms

server-medium-tests failed identically on both platforms: every EXIF-derived test returned the same fallback date and empty tag arrays. Root cause: the test factory's *testAssetsDir* resolves into *e2e/test-assets*, a git submodule (*immich-app/test-assets*) that the checkout step wasn't fetching for that job. A one-line fix (*submodules: 'recursive'*) on both branches resolved it completely. The identical failure signature on both platforms is itself a useful control: it confirms the two benchmark branches really were running the same workload.

6 Reliability

Platform	Job-runs attempted	Failures	Flaky (fail → pass, no change)	Cache inconsistencies
GitHub Actions	14	1	0	0
Blacksmith	14	1	0	0

Tally excludes the deliberate failure tests, which were intentional and are covered under Observability.

Both platforms recorded exactly one failure — and it was the **same** failure: the server-medium-tests submodule bug described in Section 5, our workflow's fault, not a platform issue. Fixed identically on both branches, then 100% success on every subsequent run. No flakiness was observed on either platform: no identical rerun producing a different result, no cache-corruption symptom, no run that queued and never started (once the Blacksmith App was connected).

Honest limitation: 14 job-runs per platform is enough to say “no flakiness observed,” not “provably equally reliable.” Real flakiness is intermittent and typically needs the original plan's 20–30 runs per cell to detect with confidence. This dimension is a tie *at the sample size actually collected*, not a proven tie.

7 Observability & Debugging

A deliberate failure (one flipped assertion in `server/src/utls/mime-types.spec.ts`) was pushed to both branches to capture the real debugging experience on each platform.

What was the same — which is most of it

- **Log clarity:** identical and excellent on both. Vitest's own reporter prints the exact file:line, an inline code snippet, and a clean expected/received diff (*Expected: “.pngx” / Received: “.png”*) straight into the raw Actions log. The log content comes from the test runner, not the platform — a faster platform does not mean better logs, and here it genuinely does not differ.
- **Per-step timing:** natively visible on both — Blacksmith runs are still standard GitHub Actions runs, same UI, same step breakdown. Blacksmith is a runner substitution, not a different CI product.
- **Time-to-signal:** a failing run took as long as a passing one on both platforms (3m29s GHA / 88s Blacksmith), because the job design has no fail-fast between steps and Vitest ran all 2,237 tests before reporting. That is a job-design choice inherited from Immich's own CI, not a platform limit — Blacksmith surfaces the failure sooner only because the whole job is faster.

Where they differ

- **SSH-into-runner debugging:** Blacksmith's dashboard advertises this as a first-class feature; GitHub Actions has no native equivalent (it needs a marketplace action such as `mxschmitt/action-tmate` added ahead of time). **Not tested in this session** — nothing failed for a non-obvious reason — so it stands as a

real but unverified asymmetry.

- **Cache hit/miss visibility:** *actions/setup-node*'s cache step logs a clear “Cache restored from key” / “Cache not found” line; *useblacksmith/cache*'s output was less chatty and did not print an obvious equivalent. A minor point, worth a closer look if cache diagnostics matter.

Net finding: the debugging experience itself is essentially identical, because both are GitHub Actions under the hood. Blacksmith's differentiation in this dimension is speed and (on paper, untested) native SSH access — not how failures are reported.

8 Overall Developer Experience

Objective observations first; the one genuinely subjective call is left unscored.

Question	What the evidence says
Which is faster?	Not a feeling — Blacksmith was faster on every single job tested, from ~1.4× (e2e) to 5.3× (the ESLint step). Consistent, not a one-off.
Which requires less operational effort?	GitHub Actions, clearly — the zero-setup default. Blacksmith needed a GitHub org, an overnight human approval of its App install, and a workflow-editing pass to swap runner labels and cache/build actions.
Which inspires more confidence?	Split. GitHub's availability is unconditional — no approval gate, no third-party support queue. But once Blacksmith was running, it was exactly as reliable as GitHub in this session. The confidence gap is about getting started, not about running.
Which has better documentation?	Mixed. Blacksmith's quickstart and migration guide were clear and accurate on the mechanics, but omitted the org-ownership requirement, and its Migration Wizard's rewrite-everything default is wrong for a controlled comparison.
Which is better day-to-day?	Once a job is running, essentially identical — same Actions log UI, per-step timing, and failure reporting. Blacksmith's advertised SSH debug and analytics went unneeded (and unverified) this session.

Which would I choose? That is the one genuinely personal call in this report, and it is deliberately not answered here. The evidence is what it is: substantial, consistent speed gains — most pronounced on CPU-bound work like linting — purchased with real, mostly one-time setup friction that GitHub Actions does not ask for at all.

9 Final Scorecard

Dimension	GitHub Actions	Blacksmith	Winner	Notes
1. Setup & Onboarding	Zero setup, always available	Org required, overnight approval wait, workflow edits needed	GitHub Actions	Blacksmith's friction is real and mostly one-time, but it was the single biggest time cost of the whole session
2. Workflow Performance	Baseline	1.4–5.3× faster across every job tested; biggest edge on CPU-bound ESLint	Blacksmith	n=1–2 per cell, directional not statistically powered — but the pattern was consistent across all 6 jobs
3. Reliability	0 platform failures / 14 job-runs	0 platform failures / 14 job-runs	Tie	Genuinely symmetric at this sample size; both had exactly 1 failure and it was our own workflow bug, identical on both sides
4. Observability & Debugging	Identical log/UI experience; no native SSH debug	Identical log/UI experience; dashboard advertises SSH debug + analytics (untested)	Tie, edge to Blacksmith on paper	Same underlying GitHub Actions UI either way; Blacksmith's extra tooling is a real but unverified claim
5. Overall Developer Experience	Simpler to start, slower to run	Harder to start, faster to run	Depends on what you're optimizing for	Not scored — see Section 8; the choice is the reader's to make

10 What a Fuller Study Would Add

This pilot covers the full six-job matrix with honest, traceable data — every timing links to a real run URL — but at n=1–2 per cell it is directional by design. The original plan called for 20–30 runs per cell. Running it would add:

- **Statistical power.** p50/p95 durations per job and cache state, confidence intervals on the speedup ratios, and enough repetitions to detect intermittent flakiness — the one reliability property this sample size cannot rule out.
- **A true migration-effort measurement.** This pilot's +70-line from-scratch bench.yml understates real migration cost; the fuller study should port Immich's production test.yml line-by-line and clock it.

- **Verification of the unverified claims.** Blacksmith's SSH-into-runner debugging and cache/test analytics were advertised but never exercised here — a fuller study should trigger a non-obvious failure and actually use them.
- **A longer reliability horizon.** Multi-day operation across queue-pressure conditions, rather than a single evening's runs.

Until then, the honest one-line summary stands: **Blacksmith was consistently and sometimes dramatically faster on this real-world workload — most of all on CPU-bound linting — at the cost of setup friction GitHub Actions simply does not have; reliability and debugging were indistinguishable at pilot scale.**

Source data: results/pilot-notes.md and results/pilot-results.csv (28 recorded job-runs with run IDs and URLs), Wanderment-Farms/immich fork, benchmark session of July 8, 2026.